

RECLAIM 122

Product Requirements Document

MVP v1.0

PRODUCT THESIS

A mobile-first, Apple-inspired personal consistency tracker for a focused 122-day mission: reclaim discipline, improve physical fitness, build a Product Management career, and become employable in Python.

Product	RECLAIM 122
Mission window	September 1, 2026 – December 31, 2026 (122 days)
MVP audience	Single primary user; architecture prepared for future multi-user use
Primary platforms	Mobile Safari + desktop browsers; responsive PWA
Repository strategy	Public showcase/demo repository + private personal deployment/repository
Document status	Finalized MVP specification

1. Executive Summary

RECLAIM 122 is a personal web application designed around a simple behavioral loop: show up, record what happened, see the streak, and continue. It is intentionally not a large productivity platform. Its MVP focuses on four goals and makes progress visible through calendars, streaks, completion counts, and lightweight analytics.

The product is inspired by the clarity and visual polish of Apple Health, but it is not intended to replicate Apple's product. RECLAIM 122 should feel calm, premium, simple, touch-friendly, and immediately understandable. The core experience should take less than two minutes per day and ideally closer to 30–60 seconds.

CORE PRINCIPLE

The app must measure consistency without turning consistency into a judgment. A bad day is data, not a personal verdict.

2. Product Vision & Objectives

2.1 Vision

Build a personal performance system that makes progress visible, reduces friction in daily check-ins, preserves an honest history of behavior, and helps the user remain consistent through a fixed 122-day mission.

2.2 Four Mission Goals

Goal	Purpose	MVP Green Rule	Weight (future analytics)
Discipline	Remain porn-free and resist intentional behavior that conflicts with the mission.	No intentional porn viewing and masturbation on that day.	30%
Fitness	Show up for physical training and maintain basic activity tracking.	At least one workout/activity session.	25%
Product Management	Create daily forward motion toward a PM role.	At least one meaningful PM action, e.g. one application.	25%
Python	Build employable Python capability through consistent learning.	Any meaningful Python progress, e.g. learning a topic or practice.	20%

Important: the weights are retained as a product-design decision for future analytics. The MVP does not calculate or display a daily 0–100 score.

3. Problem Statement

Generic habit trackers often reduce behavior to a binary checkbox and can create unnecessary pressure when a user misses one area. RECLAIM 122 addresses a different problem: how to keep four important life goals visible at the same time while preserving psychological momentum.

- The user needs a single visual record of a fixed 122-day mission.
- The user wants the calendar and streak to be motivating enough to make opening the app worthwhile.
- The user needs independent visibility into each goal without turning the daily check-in into a form.
- The system must distinguish a genuinely failed day from an unrecorded day or an excused day.
- The user wants to access the tracker from both phone and computer without maintaining a spreadsheet.
- The public version must be safe to showcase without exposing the user's private tracking data.

4. Product Principles

Principle	Requirement
Fast	Daily check-in should normally take under 2 minutes, ideally 30–60 seconds.
Simple	Use taps, toggles, small numeric inputs, and optional notes. Avoid long forms.
Visual	The calendar, streaks, and completion counts are first-class UI elements.
Non-judgmental	The app reports reality without shaming, celebrating excessively, or insulting the user.
Honest	Not recorded is not automatically treated as failure.
Flexible	Any previous day can be edited at any time.
Mobile-first	The phone experience is primary; desktop expands the same information.
Private by design	Sensitive personal tracking data must never be placed in the public repository or public demo dataset.

5. Status & Streak Model

5.1 Goal Statuses

- **GREEN** — The user showed up and completed the minimum meaningful action for that goal.
- **BLUE** — Optional user-selected status when the user believes the day was partial or below their intended standard. Blue must never be required.
- **RED** — The goal's explicit failure condition occurred.
- **YELLOW** — The user marks the goal/day as excused because circumstances prevented normal participation.
- **WHITE/NOT RECORDED** — No entry has been recorded. This is not a failure.

5.2 Overall Day Status

Each day also has one overall mission status. If any of the four goals is RED, the overall day is RED. Otherwise, the user can use GREEN/BLUE/YELLOW according to the recorded goal outcomes. A day with no records remains NOT RECORDED.

5.3 Discipline Failure Rule

Urges do not create a failure. An urge is not fully controllable and therefore is not treated as a negative event. For MVP purposes, Discipline is RED only when the user intentionally watched pornography and masturbated that day. The app should not ask the user to log urges unless that is added later.

5.4 Streak Rule

The main streak continues through GREEN, BLUE, and YELLOW days. RED breaks the streak. NOT RECORDED does not silently count as success; the UI should clearly distinguish an unrecorded day from a completed day.

The same rule applies to each individual goal's streak: GREEN, BLUE, and YELLOW continue the goal streak; RED breaks it.

6. Daily Tracking Specification

6.1 Discipline

- Primary input: Porn-free status.
- GREEN: no intentional pornography viewing + masturbation that day.
- RED: intentional pornography viewing + masturbation that day.
- Urges are not scored and do not cause RED.
- No requirement to disclose explicit details beyond the status.

6.2 Fitness

- Workout: simple completed/not completed control. Any meaningful workout/session qualifies for GREEN.
- Steps: optional numeric entry.
- Weight: optional numeric entry; no mandatory daily weigh-in.
- The MVP does not calculate calories, protein, macros, sleep, body-fat percentage, exercise sets/reps, or strength progression.

6.3 Product Management

- Minimum meaningful action qualifies for GREEN.
- Examples: one job application, meaningful PM study, networking, portfolio work, interview preparation.
- MVP stores application count as a number only.
- Individual company/job records are intentionally out of scope for MVP.

6.4 Python

- Track learning time only.
- Any meaningful progress qualifies for GREEN: a new topic, practice, problem solving, tutorial, or project work.
- MVP does not track problem counts or topic taxonomy.

7. Daily Check-In UX

The check-in is the highest-friction risk in the product and must therefore be designed around speed. The user can log items throughout the day and then finalize the day later.

7.1 Fast Check-In

1. Open Today.
2. Tap the four goal controls or update the small numeric fields.
3. Optionally enter steps, weight, and PM application count.
4. Optionally add a short note.
5. Tap Complete/Save Day.
6. The system derives the overall status and updates the calendar/streaks.

7.2 Editing

Any previous day can be edited at any time. Changes must immediately recalculate affected statuses, streaks, and analytics.

7.3 Yellow / Excused

The user can mark a goal or day as Yellow at any point, including after the day has passed. The system should offer a short reason selector such as Travel, Illness, Family, Work, Emergency, Other. A short reason is enough; no essay is required.

7.4 Notes

Notes are optional. They must never be required to complete a daily check-in.

8. Calendar & Progress Experience

The calendar is the visual centerpiece of RECLAIM 122. It should provide an immediate sense of continuity, streak, and accumulated effort.

8.1 Mission Calendar

- Shows the overall status for each day.
- Supports month navigation across the RECLAIM 122 mission window.
- Each day is tappable/clickable.
- Selecting a day opens its detailed record.
- Future dates remain visually distinct and cannot be confused with failures.

8.2 Goal Calendars

Under the overall mission calendar, provide separate calendar views for Discipline, Fitness, Product Management, and Python. These are ordered according to the product's weighting: Discipline (30%), Fitness (25%), PM (25%), Python (20%). The goal calendars show each goal's own daily status and streak.

8.3 Calendar Legend

Status	Color	Meaning
GREEN	Green	Meaningful progress / completed

BLUE	Blue	Optional partial-day designation
RED	Red	Goal/day failure
YELLOW	Yellow	Excused / unavoidable circumstances
NOT RECORDED	Neutral/white/gray	No record exists

9. MVP Screen Specification

1. Home / Mission Dashboard

- RECLAIM 122 identity and current mission day.
- Current streak, longest streak, completed/total days, and remaining days.
- Overall monthly calendar or calendar entry point.
- Four goal summary cards/statuses.
- Primary Today / Check-in action.
- Clean Apple-inspired card hierarchy; no motivational shouting.

2. Today / Daily Check-In

- Four goal controls.
- Fitness: workout, optional steps, optional weight.
- PM: application count and simple progress control.
- Python: learning time.
- Optional note.
- Save/finalize action.
- Very low interaction count and thumb-friendly controls.

3. Calendar

- Overall mission calendar.
- Separate goal calendars.
- Tap a day to inspect/edit it.
- Month navigation.
- Legend and streak summary.

4. Goal Detail

- Goal-specific current streak.
- Longest streak.
- Successful/partial/failed/excused/not-recorded counts.
- Goal calendar.
- Simple trend/consistency information.

5. Analytics

- Overall completion counts.

- Current and longest streaks.
- Goal-level completion counts.
- Simple monthly/mission consistency trends.
- No daily score.

6. Settings

- Light/dark appearance.
- Mission configuration scaffold for future RECLAIM periods.
- Data export/import for resilience.
- Private/public environment indicator if needed.

10. MVP Analytics

- Completed/total days.
- Current overall streak.
- Longest overall streak.
- Green, Blue, Red, Yellow, and Not Recorded counts.
- Current and longest streak per goal.
- Goal completion counts.
- Optional simple monthly consistency visualization.
- No weighted daily score in MVP.
- No advanced behavioral correlation or AI analysis in MVP.

11. Proposed Data Model

The architecture should support future RECLAIM periods even though the MVP is focused on RECLAIM 122.

Entity	Key fields	Purpose	Notes
mission	id, name, start_date, end_date, timezone, status	Defines a RECLAIM period	MVP seeds RECLAIM 122
day_records	id, mission_id, date, overall_status, note, updated_at	One daily record per mission date	Unique mission_id + date
goal_records	id, day_id, goal, status, metadata	Stores the four goal statuses and goal-specific fields	Unique day_id + goal
fitness_data	goal_record_id, steps, weight, workout_done	Fitness metrics	Optional values
pm_data	goal_record_id, application_count	PM numeric data	MVP only
python_data	goal_record_id, learning_minutes	Python numeric data	MVP only
excuse_data	day_id/goal_record_id,	Yellow reason	Optional

reason

12. Recommended Technology Stack

Recommended architecture: a static, installable web app with a managed cloud database/auth layer. This balances the user's requirement for GitHub-based hosting, mobile Safari access, cross-device persistence, a strong portfolio codebase, and low operational overhead.

Layer	Recommendation	Reason
Frontend	React + TypeScript + Vite	Fast development, strong ecosystem, type safety, easy static deployment.
UI	Tailwind CSS + shadcn/ui/Radix primitives, customized heavily	Fast accessible components while allowing a custom iOS-inspired visual system.
Icons	Lucide React	Clean, consistent iconography; avoid copying Apple's proprietary icon assets.
Charts	Recharts or lightweight SVG/CSS charts	Enough for simple MVP analytics without a large visualization dependency.
PWA	vite-plugin-pwa	Installable app shell, service worker, caching, mobile-first experience.
Database	Supabase Postgres	Managed relational database with browser-friendly SDK and strong authorization via RLS.
Auth	Supabase Auth, passwordless magic link for private deployment	Minimal-friction cross-device identity; no username/password workflow.
Hosting	GitHub Pages via GitHub Actions	Fits the user's public/private repository showcase strategy and static frontend architecture.
Source control	Git + GitHub	Public portfolio version and private personal version from the same codebase.
Testing	Vitest + React Testing Library; Playwright for critical flows	Protects status/streak logic and verifies mobile/desktop user flows.
Validation	Zod	Validate stored/input data and keep frontend assumptions explicit.

IMPORTANT ARCHITECTURE DECISION

GitHub Pages alone is static hosting. It does not provide cross-device application data synchronization. Because the requirement is phone + computer access with persistent history, RECLAIM 122 should use a cloud database such as Supabase. The public GitHub repository must contain code only, never private tracking data or privileged secrets.

12.1 Authentication Interpretation

The user does not want a traditional account-management experience. For the private deployment, the recommended compromise is passwordless magic-link authentication: the user identifies themselves only when needed, and the browser keeps the session. This preserves cross-device synchronization without forcing a username/password system. The public portfolio/demo deployment should use synthetic sample data and must not expose the private database.

13. Deployment & Repository Strategy

7. Create a clean React/TypeScript/Vite codebase.
8. Create a public GitHub repository containing the application source, README, screenshots, architecture notes, and synthetic demo data only.
9. Create a separate private repository or private deployment configuration for personal tracking.
10. Create a private Supabase project for real tracking data.
11. Use Row Level Security so only the authenticated user's rows can be read or modified.
12. Store only publishable frontend configuration in the client; never expose Supabase secret/service-role credentials.
13. Deploy the static frontend to GitHub Pages through GitHub Actions.
14. Configure the PWA manifest, icons, service worker, and correct GitHub Pages base path.
15. Test the private deployment from iPhone Safari and desktop Chrome/Edge before considering MVP complete.

14. Privacy & Security Requirements

- Sensitive personal tracking data must not be committed to Git.
- No hard-coded private data in source code.
- No service-role or secret database keys in browser code.
- Database tables must use Row Level Security.
- Public demo data must be synthetic.
- Exported private data must be treated as sensitive.
- Do not collect unnecessary analytics or third-party tracking in MVP.
- Do not send personal goal data to AI services in MVP.

15. MVP Acceptance Criteria

- The app opens correctly on iPhone Safari and desktop browsers.
- The user can view the current RECLAIM 122 mission day and progress.
- The user can create/update a daily record in under two minutes.
- The user can update entries throughout the day before finalizing.
- The user can edit any previous day.
- Discipline RED occurs only under the defined failure condition.
- Fitness can record workout, optional steps, and optional weight.

- PM can record an application count and/or meaningful PM completion.
- Python can record learning time and meaningful completion.
- The user can mark Yellow and provide a short reason.
- Unrecorded dates remain Not Recorded and are not silently treated as failure.
- Any RED goal causes the overall day to be RED.
- GREEN, BLUE, and YELLOW preserve the applicable streak; RED breaks it.
- The overall calendar shows day status.
- Separate calendars show each goal's status.
- Current streak, longest streak, completed/total days, and basic goal counts update correctly.
- Data persists across refreshes and across the user's phone and computer through the private backend.
- Public demo contains no private tracking data.
- Light and dark mode work without compromising readability.
- The interface feels polished, calm, and Apple-inspired rather than like a generic admin dashboard.

16. Explicit MVP Non-Goals

- Daily 0–100 scoring.
- AI coaching or motivational messages.
- Push notifications.
- Advanced relapse-pattern analytics.
- Body transformation photo timeline.
- Calories/macros/protein/sleep tracking.
- Individual job application CRM.
- Python problem/topic taxonomy.
- Social sharing, leaderboards, XP, badges, or gamification.
- Multi-user collaboration.
- Complex journaling.
- Native iOS/Android application.

17. Post-MVP Roadmap Candidates

- Personalized but non-judgmental home-screen insights.
- Advanced correlations between habits and outcomes.
- Transformation photos and body measurements.
- PM application funnel analytics.
- Python competency map.
- Optional reminders/notifications.
- Import/export and backup improvements.
- Optional weighted Mission Progress metric that never replaces the four goal statuses.
- Reusable RECLAIM templates for 30/60/90/122/180-day missions.

18. Visual & Interaction Direction

- Apple Health-inspired information hierarchy: rounded cards, generous whitespace, restrained typography, subtle separators, clear hierarchy.
- Light mode is the primary reference; dark mode is optional but supported.
- Use a restrained palette: neutral surfaces with green/blue/red/yellow status colors.
- Touch targets should be comfortable on iPhone.
- Use bottom navigation or similarly thumb-friendly navigation on mobile.
- Avoid dense tables and desktop-admin patterns.
- Animations should be subtle and functional: status changes, calendar selection, completion confirmation.
- Do not overuse gradients, glow, glassmorphism, or gamified visual effects.
- Do not imitate Apple's proprietary logos, assets, or exact screens; borrow the design principles, not protected brand assets.

19. Portfolio / Product Management Value

RECLAIM 122 is also intended to serve as a real product-management portfolio project. The public repository and documentation should demonstrate problem definition, user-centered requirements, MVP prioritization, trade-offs, data modeling, UX decisions, technical architecture, testing, deployment, and iterative product development.

- Public README should explain the problem, product thesis, MVP, screenshots, architecture, and roadmap.
- Maintain a concise decision log explaining why scoring was removed from MVP, why daily check-in is intentionally short, and why the calendar/streak is the core experience.
- Use synthetic demo data to demonstrate analytics without exposing personal information.
- After real usage, document what changed in V2 and why.

20. BUILD PROMPT FOR CLAUDE CODE / CLAUDE

The following prompt is intentionally detailed. It can be copied into Claude Code or another coding agent as the initial implementation brief. It restates the product decisions and instructs the agent to build the MVP rather than over-engineer it.

You are the lead product engineer responsible for building RECLAIM 122, a production-quality MVP web application.

IMPORTANT: Do not turn this into a generic habit tracker, productivity dashboard, or over-engineered enterprise application. The product is intentionally small, fast, visual, mobile-first, and psychologically non-judgmental.

=====

1. PRODUCT

=====

Name: RECLAIM 122

Mission:

RECLAIM 122 is a 122-day personal consistency tracker running from September 1, 2026 through December 31, 2026.

The user's four mission goals are:

1. Discipline - stay porn-free.
2. Fitness - show up for physical fitness.
3. Product Management - make daily progress toward getting a PM job/internship/associate role.
4. Python - build enough Python skill to become employable if PM does not work out.

The philosophy is:

SHOW UP → CHECK IN → SEE PROGRESS → REPEAT.

The app should help the user reclaim focus, discipline, health, career momentum, and future opportunity over 122 days.

=====

2. CORE PRODUCT PHILOSOPHY

=====

The calendar, streaks, and completed/total days are the primary motivational mechanism.

Do NOT make the product feel like a Google Form.

Daily check-in must normally take under 2 minutes and ideally 30-60 seconds.

The app must not shame the user.

Do not display messages such as:

- "You failed."
- "You suck."
- "You broke your life."
- "Amazing, you're a beast!"
- "You fucking did it!"

Keep the tone calm, factual, encouraging, and neutral.

A bad day is data, not a personal verdict.

=====

3. TECHNOLOGY STACK

=====

Use:

Frontend:

- React
- TypeScript
- Vite

UI:

- Tailwind CSS
- shadcn/ui and/or Radix primitives where useful
- Lucide icons

Charts:

- Prefer lightweight CSS/SVG for simple analytics.
- Recharts is acceptable if needed.

PWA:

- vite-plugin-pwa
- Installable on iPhone Safari and desktop browsers.

Backend/data:

- Supabase Postgres
- Supabase Auth

Authentication:

- Use passwordless magic-link authentication for the private deployment.
- Do NOT build a traditional username/password account-management system.
- The user wants cross-device persistence without a heavy login experience.

Hosting:

- GitHub Pages through GitHub Actions for the static frontend.

Important:

GitHub Pages is static hosting. It cannot itself provide cross-device persistent application data.

Use Supabase for the data layer.

Security:

- Enable Row Level Security on every exposed Supabase table.
- All user data must be scoped to the authenticated user's ID.
- Never put a Supabase secret/service-role key in browser code.
- Only use the public/publishable client key in frontend code.
- Never commit personal tracking data to Git.

=====

4. REPOSITORY STRATEGY

=====

The codebase should be suitable for:

A. Public portfolio/demo repository

- Synthetic sample data only.
- No private personal tracking data.
- Strong README.
- Screenshots.
- Architecture documentation.
- Setup instructions.
- Portfolio-friendly code.

B. Private personal deployment/repository

- Same application code.
- Private Supabase project.
- Real personal tracking data.

- Private environment configuration.

Use environment variables for Supabase URL and publishable key.

=====

5. VISUAL DESIGN

=====

The design should be inspired by the principles of Apple Health:

- clean
- premium
- calm
- rounded cards
- generous whitespace
- strong typography hierarchy
- subtle separators
- restrained colors
- smooth but minimal animation
- excellent touch targets
- clear information hierarchy

Do NOT copy Apple's proprietary UI exactly.

Do NOT use Apple's proprietary assets or logos.

Light mode is the primary design direction.

Dark mode should be supported.

The app should feel like a polished iOS-inspired personal health/performance app, not a SaaS admin dashboard.

Mobile-first is mandatory.

Desktop should be responsive and make better use of horizontal space without changing the core information architecture.

=====

6. MVP SCREENS

=====

Build these screens:

1. HOME / MISSION DASHBOARD
2. TODAY / DAILY CHECK-IN
3. CALENDAR
4. GOAL DETAIL
5. ANALYTICS
6. SETTINGS

Do not add extra product areas unless they are technically required.

=====

7. HOME / MISSION DASHBOARD

=====

The home screen should prioritize:

RECLAIM 122

Current mission day:

DAY X / 122

Days remaining:

X

Current streak:

X days

Longest streak:

X days

Completed days:

X / 122

Then show the current month's overall calendar or a prominent calendar entry point.

Then show four goal cards:

Discipline

Fitness

Product Management

Python

Each card should show:

- current status for today
- current streak
- completed/meaningful days count

Primary CTA:

CHECK IN / CONTINUE TODAY

The dashboard should not be cluttered.

=====

8. DAILY CHECK-IN

=====

The user can log progress throughout the day and finalize later.

The UI must be fast.

DISCIPLINE:

- Porn-free: Yes / No
- Do NOT track urges in MVP.
- Urges do not create failure.

Discipline rules:

GREEN = no intentional pornography viewing + masturbation that day.

RED = intentional pornography viewing + masturbation that day.

FITNESS:

- Workout: Done / Not Done
- Steps: optional numeric field
- Weight: optional numeric field

Any meaningful workout/session qualifies for GREEN.

Do not require steps or weight for GREEN.

PRODUCT MANAGEMENT:

- Meaningful PM progress: Done / Partial / Not Done
- Application count: numeric field

- One meaningful PM action is enough for GREEN.

Examples:

- one application
- PM study
- networking
- portfolio work
- interview preparation

PYTHON:

- Meaningful Python progress: Done / Partial / Not Done
- Learning time in minutes: numeric field
- Any meaningful progress qualifies for GREEN.

Examples:

- learning a new topic
- practice
- solving a problem
- tutorial
- project work

NOTES:

- Optional text area.
- Never required.

YELLOW:

- User can mark a goal/day as excused.
- Reasons:
 - Travel
 - Illness
 - Family
 - Work
 - Emergency

Other

- Yellow can be added at any time.
- Previous days can also be changed to Yellow.

SAVE / COMPLETE:

- Persist immediately.
- Recalculate day status and streaks.

=====

9. BLUE STATUS

=====

Blue is intentionally optional.

The user may select Blue when they personally consider the day's work partial or below their intended standard.

Do not automatically force Blue based on numeric thresholds.

The purpose of Blue is to preserve nuance.

=====

10. OVERALL DAY STATUS

=====

Each day has an overall status.

Rules:

1. If ANY of the four goals is RED:

overall day = RED.

2. If no goal is RED:

overall day can be GREEN, BLUE, or YELLOW based on the recorded goal states/user designation.

3. If there is no meaningful record:

overall day = NOT RECORDED.

Do not automatically mark an unrecorded day as RED.

=====

11. STREAK RULES

=====

Overall streak:

- GREEN continues streak.
- BLUE continues streak.
- YELLOW continues streak.
- RED breaks streak.
- NOT RECORDED must not be silently counted as a successful day.

Individual goal streak:

- GREEN continues.
- BLUE continues.
- YELLOW continues.
- RED breaks.
- NOT RECORDED is not success.

Track:

- current streak
- longest streak
- completed/meaningful days

- red days
- blue days
- yellow days
- not-recorded days

Important:

Do not create a daily 0-100 score.

=====

12. CALENDAR

=====

The calendar is the centerpiece.

Overall Mission Calendar:

- Show one status color per day.
- Month navigation.
- Show September–December 2026.
- Future days should be visually distinct.
- Not-recorded days should not look like failures.
- Tapping a day opens its record.

Below/alongside the overall calendar, provide separate goal calendars:

1. Discipline
2. Fitness
3. Product Management
4. Python

Order them using the future analytics weights:

Discipline 30%

Fitness 25%

PM 25%

Python 20%

The weights do NOT create a daily score in MVP.

=====

13. ANALYTICS

=====

MVP analytics should be basic.

Overall:

- completed/total days
- current streak
- longest streak
- green count
- blue count
- red count
- yellow count
- not-recorded count

Per goal:

- current streak
- longest streak
- meaningful/completed days
- red days
- blue days
- yellow days
- goal calendar

Fitness:

- optional simple weight trend if enough weight data exists
- workout completion count
- steps entries

PM:

- total application count across recorded days

Python:

- total learning minutes across recorded days

Do not build:

- AI analysis
- relapse correlations
- advanced behavioral statistics
- weighted daily score
- complicated dashboards

=====

14. GOAL WEIGHTS

=====

Store these in configuration for future analytics:

Discipline = 30

Fitness = 25

Product Management = 25

Python = 20

Do not expose them as a score.

15. EDITING

Any previous day can be edited.

When a previous day changes:

- recalculate its overall status
- recalculate affected streaks
- recalculate analytics
- update calendar immediately

There is no seven-day editing limit.

16. DATA MODEL

Design the database so the architecture can support future RECLAIM missions.

Suggested structure:

missions

- id
- name
- start_date
- end_date
- timezone
- status
- created_at

day_records

- id
- mission_id
- user_id
- date
- overall_status
- note
- created_at
- updated_at

goal_records

- id
- day_id
- user_id
- goal
- status
- created_at
- updated_at

fitness_data

- goal_record_id
- workout_done
- steps
- weight

pm_data

- goal_record_id
- meaningful_progress
- application_count

python_data

- goal_record_id
- meaningful_progress
- learning_minutes

excuse_data

- day_id or goal_record_id
- reason

Use appropriate unique constraints, foreign keys, indexes, and check constraints.

A mission/date should not have duplicate day records.

A day should not have duplicate goal records.

=====

17. SECURITY

=====

Implement Supabase Row Level Security.

Every private row must belong to the authenticated user.

Users can only:

- read their own records
- create their own records
- update their own records
- delete their own records where appropriate

Never rely on frontend checks alone.

Never expose service-role keys.

Do not store private data in localStorage as the source of truth.

You may use local caching for offline UX, but Supabase is the source of truth.

=====

18. PWA / MOBILE

=====

The app must:

- be responsive
- work well in iPhone Safari
- support Add to Home Screen
- have a web app manifest
- have appropriate icons
- have a service worker
- provide sensible offline behavior

Offline behavior:

- If the user loses connectivity while editing, avoid data loss.
- Queue or locally persist unsynced changes if feasible.
- Clearly indicate sync state.
- Do not silently claim data is synced when it is not.

=====

19. UX DETAILS

=====

Use thumb-friendly controls.

Avoid:

- large forms
- excessive text
- unnecessary confirmation dialogs
- too many screens for one check-in
- excessive animation
- gamified badges
- loud motivational copy

Prefer:

- segmented controls
- toggles
- chips
- compact numeric inputs
- cards
- swipe/tap-friendly calendar cells
- one obvious save/complete action

The user should be able to open the app, understand today immediately, update the four goals, and leave.

=====

20. EMPTY STATES

=====

Before any data exists:

Show:

RECLAIM 122

DAY 1 / 122

Start your first day.

Do not show fake personal history in the private deployment.

The public demo may use a clearly labeled synthetic dataset.

=====

21. ERROR HANDLING

=====

Handle:

- network failure
- Supabase authentication expiry
- database errors
- duplicate records
- invalid numeric values
- offline edits
- failed sync

Never lose entered data silently.

Use calm error messages.

=====

22. TESTING

=====

Write tests for the core business logic before polishing the UI.

At minimum test:

- discipline green/red rule

- overall day red if any goal red
- blue does not break streak
- yellow does not break streak
- red breaks streak
- not-recorded is not automatically red
- editing previous days recalculates streaks
- current streak calculation
- longest streak calculation
- goal-specific streak calculation
- calendar status calculation
- date boundaries for September 1–December 31, 2026

Use:

- Vitest
- React Testing Library
- Playwright for key end-to-end flows

=====

23. IMPLEMENTATION ORDER

=====

Do NOT try to build everything at once.

Build in this order:

PHASE 1

- project scaffold
- routing
- design system
- responsive layout
- theme system

PHASE 2

- data model
- Supabase integration
- auth
- RLS

PHASE 3

- daily record logic
- goal status logic
- streak logic
- date logic

PHASE 4

- Today screen
- Home dashboard
- calendar

PHASE 5

- goal calendars
- analytics
- settings

PHASE 6

- PWA
- offline/sync behavior
- GitHub Pages deployment

PHASE 7

- tests
- accessibility

- responsive QA
- polish

=====

24. PRODUCT QUALITY BAR

=====

Before declaring MVP complete, verify:

- It looks good on a current iPhone Safari viewport.
- It looks good on desktop.
- Check-in takes less than two minutes.
- Calendar is immediately understandable.
- Current streak is obvious.
- Completed/total days are obvious.
- A RED day is visible without being emotionally aggressive.
- No score exists.
- No unnecessary features exist.
- Previous days can be edited.
- Yellow can be applied retroactively.
- Not-recorded days are not automatically red.
- Cross-device data persists.
- Private data is protected.
- Public repository contains no private data.
- The README explains the product and architecture.
- The code is clean enough to show in a PM/software portfolio.

=====

25. IMPORTANT PRODUCT DECISION

=====

Do not add features just because they are technically interesting.

If a feature does not make the daily loop faster, clearer, or more useful, leave it out.

The MVP is successful if the user wants to open it every day because:

- the calendar is satisfying to look at
- the streak is visible
- progress is easy to understand
- check-in is fast
- the four goals remain visible
- the system does not make the user feel judged

=====

26. STARTING INSTRUCTION

=====

Before writing large amounts of code:

1. Inspect the repository.
2. Propose the final folder structure.
3. Confirm the implementation plan against this specification.
4. Identify any technical contradiction, especially around GitHub Pages + Supabase + PWA + cross-device persistence.
5. Then implement incrementally.
6. After each major phase, run tests/build checks.
7. Do not invent requirements that are not in this specification.
8. Keep the MVP small.

Build RECLAIM 122 as a polished real product, not a demo toy.

21. Technical Reference Notes

The architecture recommendation is based on current official documentation for GitHub Pages and Supabase. GitHub Pages is a static hosting service for repository-based websites; cross-device persistence therefore requires a separate backend. Supabase provides Postgres, Auth, and Row Level Security suitable for a browser-based application.

- GitHub Pages documentation — <https://docs.github.com/en/pages>
- Supabase Row Level Security — <https://supabase.com/docs/guides/database/postgres/row-level-security>
- Supabase Authentication — <https://supabase.com/docs/guides/auth>

22. Definition of Done

RECLAIM 122 MVP is done when the user can open the application on iPhone Safari or desktop, quickly record the four daily goals, see the current and historical calendar, view overall and goal-specific streaks, edit any previous day, and retain the same private history across devices—without a daily check-in that feels like administrative work.